AI Data 实战笔记：从数据分层到多模态去重

> 一线从业者关于大模型训练数据、多模态数据处理、视频生成、data infra 和具身数据的技术讨论整理。

写在前面

本文整理自一个技术交流群里关于 AI 数据的讨论，只保留技术内容，去掉了人名、时间和组织八卦。内容是从业者的经验和转述，不是公开资料，文中数字均未经核实。遇到观点不一致的地方，文中会把几种说法并列写出。

---

1. 数据工作的分层

「做数据」在大模型团队里至少对应四种不同的工作，它们的技能要求和在链路中的位置差别很大。

1.1 模型数据研究

基础模型的研究可以分两个方向。「模型结构」研究结构能带来什么新上限、新能力；「模型数据」研究要增加或加强某个能力，该用什么样的数据。后者基本是纯研究，具体执行通常交给外包团队。

有的头部团队在基模上明确分成「模型方向」和「数据方向」两组。模型方向不一定改架构，更多是改训练策略、做数据的最后把关。这种拆分通常只出现在基模团队，其他生成模型团队一般不拆。

1.1.1 按「对什么负责」划分的三种角色

另一种常见的拆法是按负责对象划分数据链路上的角色：

角色	职责	对什么负责
数据工程	链路里所有工程部分：搭 pipeline、跑数	交付数据需求
数据刷数算法	链路里需要算法的部分：打标、用模型调 PE、微调模型来刷数	交付数据需求
基模算法（数据方向）	把控数据项目进度；研究模型要提升某项能力需要什么数据，再把需求派给工程和刷数算法	最终模型效果

按这种理解，工程和刷数算法本质上是基模算法的「外包」：基模算法愿意把哪些事交出来，它们才能做哪些事。

扩展到整个训练链路，责任划分是：

- infra 对训练稳定性、训练周期和速度负责。
- 数据岗对数据本身负责。
- 基模算法对模型效果负责。

一个流行的判断是：越能为模型效果负责（也就是背锅）的角色，地位越高。

这套划分也有争议：

- **边界并不清楚**。「某能力需要什么数据」是交叉问题，刷数算法也能碰，两边的区别更像前向验证和后向验证。实际情况是各方都在抢活证明价值。
- **归类之争**。一种看法认为「基模算法数据方向」就是大模型算法，搞数据本来就是大模型算法的本职，不应单列为分支；另一种看法认为它和数据算法是同一岗位，只是水平不同、关注点不同。
- **团队规模的影响**。只训一个模型的公司，这部分工作一两个人就能做，不需要专门团队；模型多的大厂才会拆这么细。

支持拆细的一种组织视角是：基模关注最终目标模型，刷数算法微调的模型不是最终目标，两者有本质区别。让基模算法去刷数，就像拿 B300 跑 CNN，是资源浪费；拆开后既能提高集群利用率，也能按梯度定薪。

1.2 数据算法

数据算法是桥接训练和数据的角色，核心是懂训练的人来做数据，知道模型要什么。主要工作有两类：

- 按数据的质量和量级，决定它在 pre-train、mid-train、post-train 哪一段供给，并且可以前移或后移。
- 对着 benchmark 提点。

它比「洗完就交付」高一层。和数据工程相比，对训练的理解会强很多，但差距没有想象中大，主要在认知和 scope。日常仍然是搭基建、对着 bench 提点。

这个角色是否算「训练」存在争议。一种看法是这已经是全链路训练，预训练本来就主要在做数据；另一种看法认为它本质上仍是算法组里的数据工程。

日常工作的具体形态：

- 管线通常不是数据算法搭的。数据算法盯管线同学的跑数进度，跑慢了要推动。
- 按能力缺陷调整策略。例如训出来的模型某项能力不行，就改 caption，专门强调这一点。大部分时间都在调策略。
- 调研哪类数据有价值，再推动工程去爬、去洗。
- 预训练阶段做合成数据，平衡各 domain 的数据量和配比，再用消融实验看效果。
- 训练「数据生成模型」做数据飞轮，即用 SFT 出来的模型生成数据。这算比较高级的活。

- 要同时对接上游的基模算法和下游的数据工程。在模型架构趋于稳定的当下，普遍认为数据算法比训练更累。
- 训模型的活通常轮不到数据算法，它的职责是把数据和策略搞到位。

价值取决于能不能看到实验。 数据算法如果看不到预训练实验结果，就只是在执行需求，价值有限；能看到实验，才能真正闭环。

1.3 数据工程

搭各种数据 pipeline 和测评体系，都属于 AI Data 的工作范畴。

数据工程可以粗分为获取、处理、训练三段，每段都能往深做。更细的拆法是四层：

- **基建**：数采、数据加工、搜索。数据量大时要按 domain 看，或者为补某项能力去用场内数据。基建不够用就持续迭代。
- **上层构建**：训练数据怎么组织；哪些阶段多做 caption；质量怎么层层筛；对着某个 bench 看题目分布，在不透题的前提下造题，或者把能力拆成原子能力再补数据。这一层偏沟通和协调。
- **存储**：底层存储不用自己管，要管的是数据的版本和来源（sourcing 归在获取里）。
- **可观测**：不让能力提升完全黑盒。domain 分治、tag 分治、训练配比都落在这一层。

典型的 pipeline 目标常被概括为：洗海量数据不 OOM、全局 hash 去重不出 bug、万卡训练时把万兆网卡喂满、管好几十个版本的数据资产。实际团队里，有些目标完全不涉及，有些在做但远比这句话复杂。

数据工程的价值点，一种说法是「知道什么数据能涨点，再加上有评测指标」。要注意，执行评测和设计评测不是一回事，设计评测标准的工作量要大得多。

至于数据工程对上层提供什么能力、封装了什么，一种坦率的回答是：如果搬运和打标算封装，那数据工程就是「算法的 agent」。

1.4 执行层外包

纯执行的数据工作，比如打标、清洗、调用模型和写 prompt，普遍外包。一个典型组的配比是约 3/4 做数据、1/4 做训练、再加少量 infra；纯数据执行外包出去，组内留下数据算法和训练。

1.5 数据和训练对不齐

这是反复被提到的痛点。只做训练的人对数据没有感知，上线后爆 case，很多时候是因为没看过数据。理想状态是数据和训练由同一个人做。

反过来，只做前向交付、看不到后链路的数据岗位也很难做好。数据要做好，需要拿到全链路认知；做得太靠前，视野就会少很多。预训练数据偏原子能力建设，洗得再精细，效果也有限。

即便组织上把数据和训练分开，训练岗也必须看数据：

- 一种时间分配是约 70% 跑评测、看数据，30% 做训练；数据算法则是 100% 在数据上。

- 典型的训练流程是：infra 先适配好，一键启动训练脚本；loss 有问题就回头查数据。
- 有团队的主训已经开始亲自调 caption 的 PE。一句流行的说法是「训练就是给数据擦屁股的」。
- 共识是：做算法不看数据，等于白做。

1.5.1 改架构 vs 改数据和训练策略

关于「基模算法是不是主要在改模型结构」，讨论里的看法很分化：

- **不动架构的一方。** 很多基模团队基本不动架构，主要工作是数据、训练策略和推理。改架构意味着从 0 开始重训，成本和时间都等不起。真正的主要工作是盯实验、改数据配比、找坏样本、调整训练策略。
- **署名排序作为佐证。** 有团队的技术报告里，做数据和训练的排在前面；做架构 research 的只有一个人，排在末尾，且对数据和训练策略没有感知。
- **另一种观察。** 也有公司几十人里一半在做模型结构 research，实际干活的是训练 infra、数据组全体和极少数大模型算法。批评意见认为这类预研 research 本来就不是为模型服务的，基本用不上。
- **头部数据团队的情况（转述）。** 有被认为数据做得最强的视频生成团队，转正的博士大多在洗数据，做 research 的主要是未转正的人。

一个反映「改架构成本」的案例见 5.8 节：某团队的旧模型训了很久，效果不如新的开源模型，但算法团队仍不愿换架构。

1.6 Data、Data Infra、大数据

这三个词经常被混用，但指的是不同的东西：

- **Data**：洗 LLM 或多模态训练数据，包括数据算法和数据工程。
- **Data infra**：支撑数据处理的基础设施。数据库那套能力迁移到 AI 侧，是 data infra 的一环；再加上 Spark、Ray 这类分布式调度，数据加工离不开它们。有的团队把数据管线加速、dataloader、checkpoint 也划进 infra。具身方向的 data infra 通常还要求 ROS 相关的垂直经验。
- **大数据**：传统意义的大数据平台，通常被算作 data infra。

1.7 Data agent 不是 data

data agent 指的是取数、查数用的 agent，和「数据算法」是两回事。常见形态是在数据平台上挂一个 agent，用对话生成 SQL 查数。已有的一些平台内 agent 可用性很差。

另一种设想是把获取、处理、评测、训练每个阶段交给一个子 agent，把整条数据流程自动化；目前这不是 data agent 的主流含义。

一个工程化思路是做一个通用 CLI：

- 任意 agent 都能接入，把各数据平台的 skill 串起来。

- 内置平台专用知识，例如某些平台上不能写 `save as table`。
- 自带 pipeline 和数据搬运的 demo，一句自然语言完成取数并产出 demo。
- 依托成熟文档库，串起大部分内部基建。

反对意见是：`save as table` 这类细节归各阶段的工程管，真要做到这一层需要非常庞大的封装。更轻量的替代是直接写知识库喂给 coding agent。

1.8 挖负例算不算洗数据

一种看法认为挖负例就是洗数据；另一种认为挖负例已经属于 data research，比纯数据工程更靠近研究。

---

2. 训练数据的生产

2.1 阶段不锁死，靠 bench 闭环

同一份数据不会一开始就固定在 pre、mid 或 post。根据量级、质量等因素决定放在哪一段，之后还可以迁移。

每个训练节点有自己的 bench，最终涨点看的是公开 bench，后训练阶段尤其如此。整个过程是一个闭环：

```
处理好数据 → 送去训练 → bench 提点 → 回头修改数据
```

团队之间的差别在于上游能看到多少下游。如果数据交出去之后就看不到训练结果，数据侧会被隔离得很死。

2.2 trick data 放在 mid-train

读技术报告时的一种判断是：模型能力的真功夫在预训练加 SFT，很多 trick data 放在 mid-train，RL 更像宣传。反对意见认为 RL 仍然起作用。

2.3 数据量和参数量的关系

对于预训练数据量和参数量的比例，有两种看法。

第一种参考 Chinchilla（arXiv:2203.15556）的经验值：

参数量	粗略训练 token 数
7B	约 140B
70B	约 1.4T

参数量	粗略训练 token 数
400B	约 8T

计算方式是：训练 token 数 ÷ 参数量 = tokens per parameter。

第二种看法认为数据量和参数量关系不大，主要看训练阶段。作为参照，有模型公开称预训练用了 30T token；也有模型宣称使用了 30T token 的多模态预训练语料（构成未验证）。头部团队的数据流水规模在万亿级。

2.4 为什么评测爱选 math

math 数据比较干净，而通用文本整体偏长、噪声多。这也是 math 容易「好洗」「好刷」的原因。

2.5 后训练值不值得做

不是什么场景都值得后训练，要看 ROI。研发团队一般不做后训练，因为成本高、需要 GPU；更便宜的替代方案是挂知识库，或者优化记忆系统。一个实际例子是监管报送的血缘穿透场景，最终因为没有 GPU 资源放弃后训练，改做记忆系统。

2.6 数据合成与蒸馏

预训练和后训练的数据合成是一个独立方向，招聘上通常要求多模态算法背景或相关论文。

数据供给链大致是：数据先经过筛选或合成，再由数据平台团队对接数据算法团队。

关于蒸馏有两种路线判断：

- 先蒸馏，快速把模型能力拉起来，再回头精做数据。
- 头部团队自有数据足够，不依赖蒸馏。

coding agent 的交互数据被认为是最优质的数据之一，可以直接加进训练。

2.7 一个小技巧：jsonl shuffle

最简单的 jsonl 打乱方式：

```
import random
lines = open("data.jsonl").readlines()
random.shuffle(lines)
```

数据量大到内存放不下时，这种方式就不适用了。

---

3. 评测与自进化闭环

3.1 评测集怎么沉淀

典型问题是：语音或 agent 的自进化，怎么区分是窄任务过拟合还是真的进化了？一套可行的做法如下。

评测集的来源有两部分：自己洗的数据，加上业务标注的数据。上线前统一用这套评测集。

针对特定 case，先用模型洗一遍，再按一定比例混入评测集。

评测类型主要有四类：

- bad case
- 对抗样本
- 历史回归
- 消融（用来看策略的敏感性）

rubric：尽量用线上 rubric。没有线上数据时，找产品和运营一起定。这不应该是一个人的活。

混入比例的具体数字、标注规范和 rubric 细则，讨论中没有展开。

3.2 冷启动：没数据就没 eval

冷启动阶段确实没有数据。可行的做法是：

- 上线后采集特征数据。
- 冷启动期让 agent 模拟用户去采数据。

不同场景的难度差别很大。coding、客服、金融、法律的数据量足够，答案也相对有标准；其中金融和法律大量是纯 RAG。电商搜索被认为更难。核心前提是能保证数据和语料。

loop 难做，很大程度上取决于数据集和评测标准。指标来自数据和 case，拆解无法先验；数据不够时，连指标是否合理都无法判断。这套逻辑不只适用于具身，本质上是一套标准的后训练玩法。

3.3 自进化 agent 的闭环

一个常见的闭环设计如下：

```
执行任务 → 收集轨迹 → 评价成败 → 提炼经验 → 再验证 → 更新记忆 / 技能 / 工作流 / 模型
```

实践中的限制在「评价」这一步。主观评价离不开 human check。目前能落地的做法是：人给出指标和认为好的 case，划分评价维度，让 agent 在小范围内 loop 去改 prompt，指标由评测体系提供。

3.4 RSI 相关基准

关于递归自我改进（RSI），多数讨论其实是「半环 RSI」，默认存在一个 Golden Verifier。几个把 RSI 拆成可独立评测能力的基准：

名称	思路
Aspire（arXiv 2608.31111）	把下游评测藏起来，只给模糊的能力目标
S^3Gym（arXiv 2608.31100）	拆成自测、自判、自改进三项
HarnessDev（arXiv 2609.01437）	评估 harness 的创建和演化是否真的生效

相关资源汇总见 https://github.com/zoezou2015/Awesome-Data-Centric-Recursive-Improvement 。

一个判断是：RSI 和模型自进化至少需要一套超级庞大的数据工程作为支撑，目前整体仍处在 bench 阶段。

3.5 Agent 评测框架的组成

一个 agent 评测框架的典型能力清单：

- 接入 SWE-bench、Terminal-Bench、GAIA、τ-bench 等基准。
- 任务集管理。
- 执行环境。
- 批量调度。
- 自动化评分。
- trace 回放。
- 负面案例归因。

这类系统从工程上看更接近 agent infra，而不是单纯的评测。

---

4. 专家数据与标注

4.1 专家数据的价格和标准

- 一条专家数据的价格大约在 500 到 1000 以上。
- 一条 long-horizon 数据的生产成本在一千多。

标准很乱，先要看领域：多模态视觉、理解、美学，还是法律、金融。不管哪个领域，目的都是提升模型智能。

美学类数据最难拿。它主观、拆解维度多，基本要靠垂类专家来拆。实践中能落地的美学数据，往往被限制在工程师设计和网页设计这类相对可界定的范围。

4.2 标注平台的典型流程

给大模型准备训练数据的标注平台，流程通常是：

```
采买数据 → 自己清洗一遍 → 打标签 → 送去训练
```

这类岗位如果只做前向交付、看不到后链路，工作会很痛苦。

4.3 远程打标的真实成本

远程打标平台上，一条标注标价可能到几百元，但通常要花两三个小时。其中一部分任务已经不是简单打标，而是要求标注者定制标准。

4.4 外包打标的质量问题

外包打标的质量普遍不稳定。常见的分工是：外包只做标注和调用，不写代码；写代码的是正式员工。

---

5. 多模态数据

5.1 为什么多模态数据难

三个主要难点：

- **数据分布**：code 和 text 数据最多，多模态也建立在 code 和 LLM 之上。
- **标注缺乏可解释性**：可以做一份纯图像的 GT，但人怎么打？打标的人未必比模型更懂。
- **维度太多**：还会碰到美学、品味这类主观维度。

以 OCR 为例，各家能力差不多，要看场景是 GUI 还是识别类数据。开源方案里常被问到的是百度的 PaddleOCR，但也有做多模态理解的团队并不用它。单纯识别出 bbox，和真正理解内容是两回事。多模态模型本身也要分 audio、image、video 看。「识别」和「理解」的边界怎么定，没有结论。

图像生成模型审美差的一种解释是：生成通路和 LLM 没有互通，生成侧的数据没有训进 LLM。

5.2 视频打标

数据链路里调用模型的部分，大模型和小模型都会用，可以统称为打标。视频方向做得最深。OCR、ASR 这类能力已经比较成熟，不算深的部分。

5.3 案例：图搜入库怎么「只存一份」

问题。 图搜是用 query 召回图片，视频搜是用 query 召回视频。目标是在入库前就做到每份内容只存一份。这是一个纯工程问题。

视频相对简单。 按来源站点自己的 id 去重即可，来源基本是大型视频站。这是最基本的去重，缺点是同一条内容换 id 重发会漏掉；另外要处理重复搬运。大部分场景只看大站，不太依赖公开数据集。

图片难得多。 图片没有这样的大站，非常依赖数据集。很多数据集在 Hugging Face 上，零散，下载后还要去重和重新组织。靠 meta 信息去重几乎无解。

hash 去重的争议。 一种管线是 download → save → sha256。反对意见认为 hash 或按字节数去重几乎等于没做，碰撞也不小。另一个选择是 Meta PDQ 这类感知哈希，比刷 embedding 便宜。也可以用 SSCD，或 CLIP，取决于任务。

推荐方案：两段式。

1. 先做 URL 去重。这一步去不掉「同图不同 URL」。
2. 再做图像内容去重。

存储上，图只存一份，来源信息存多条。内容去重的开销躲不开。

内容去重：embedding + 层次聚类。 入库本来就要刷 embedding，刷完直接做层次聚类，阈值卡在 0.9 或 0.95。这种方法不受分辨率变化影响。需要注意，同一类图（比如全是表格）相似度天然偏高，只有完全相同才会超过 0.9。embedding 模型可以用自己的 ViT，也可以用 Qwen reranker。经验分桶是：

相似度	判定
> 0.5	相似
> 0.9	相同

成本。 一张图如果被 n 个数据集引用，可能要做 n 次 embedding。存量数据刷一遍可能要几个月，日增量则容易处理。多模态团队往往分不到足够的算力资源。

打标方案。 不需要可解释性时，用 CLIP 即可。需要可解释性时，就多拆 tag，保证每张图的 tag 组合唯一。具体做法：

1. 把数据源的标题、简介、评论全部交给旗舰模型，提取核心词。
2. 100 万量级的图约需 10 个 tag，一亿以上约需 20 个。写法参考预训练的图文 caption。
3. 同一张图要 roll 很多次结果才稳定，多次结果取并集，再对 tag 去重。
4. 成本估算：按 5 美元 / 百万 token 的旗舰模型价格，打 tag 大约 5 到 6 万美元封顶；用自有模型则几乎不花钱。

可解释性打标的一个好处是不受自家模型能力的制约。

脏数据一例。 一批约 50 亿的 URL 中，约 20 亿来自同一个成人站点，按主站对齐后重合度很高。

数据采买。 国内有数据供应商会先做隐私计算再出售数据；做隐私计算的厂商数量很少。头部模型团队的数据采购供应商通常只有少数几家。

5.4 Caption 的新用法

有些团队的多模态数据处理流程是：先做 ASR、OCR，再给模型生成 caption，另外做实体检测和动作绑定。视频生成也需要这套流程。caption 质量很大程度上是一门手艺。

过去 caption 主要用于 prompt 工程，靠反复调 prompt 得到；现在开始出现把 caption 数据拿去做 SFT 的做法。一种 caption 格式类似「实体1 在 xxx 上看着 实体2」，动画领域已经用得很多。

caption 由谁来打存在两种看法。一种认为是大模型打标；另一种认为不是大模型，因为大模型的 CV 能力本身也是小模型训出来的。折中的理解是「人工标注 + 工具检测」。

5.5 Code data 的问题

code data 往往乱到无法 review：代码块的执行顺序、意图、为什么乱序，都完全不可预测。这部分清洗的责任更多落在工程侧。讨论中没有给出清洗方案。

5.6 其他数据需求

- 第一人称视角的优质数据，以及第一人称、第三人称、VR 数据，需求在增加。
- 纯靠「洗」解决不了多模态数据的核心问题。卡点在视觉和真实理解，本质是复杂的 if/else 判断，而不是清洗这个动作本身。因此即便是输出结构化 JSON 更稳的新型非自回归模型，也暂时难以直接用于多模态清洗。

5.7 科研侧的数据工作

学术项目里的多模态数据工作，比如组合图像检索方向，通常包括：新数据集构建、数据预处理、对比方法复现、模型训练和实验评测。

5.8 视频生成模型与生成式数据

以下是使用者的个人评测和看法，模型名称按讨论原文保留。

自动驾驶数据生成的常见术语：

- **GT**：Ground Truth，真值标注。
- **HDMap**：高精地图。
- **edge**：讨论中的解释是「视频经模型提取后得到的新数据」。在可控视频生成里，它通常指边缘图（比如 Canny 边缘），用作控制条件。

数据生成模型。 业务方训练数据生成模型，本质是用 SFT 出来的模型做数据飞轮。

案例：灌领域数据的旧模型 vs 通用开源模型。 一个团队用 wan2.1 灌了大量自驾数据，训练了很长时间，生成的自驾场景反而不如开源的 minimax-h3，而后者并没有针对自驾专门训练。即便如此，算法团队因为前期投入太大，不想换架构。

各模型的使用反馈：

- **minimax-h3**。 生成自驾场景效果强；支持输入多个视频，这一点被认为很有价值；评测了几个模型，它是其中最好的。反对意见是：另一团队测通用视频生成时，minimax 还没赢过 wan2.2（但 minimax 训得也不多）。也有人认为它生成的结果仍是「一眼 AI」。
- **开源的意义**。 此前 sd 和 wam 都是闭源的，h3 开源会造福一批做世界模型的公司。但它做不了短剧和广告素材。
- **短剧场景**。 一种看法认为目前只有 sd2.5（结合上下文应指 Seedance）能做短剧：2.5 相比 2.0 最大的进步是人物一致性，2.0 需要额外写很多 prompt 工程来弥补，有业务线已全面切到 2.5。反对意见是 2.0 时代已经有大量短剧和成熟 workflow，也有不少人用 wam 生成。
- **happy horse**：被认为不如上述模型。
- **工作流的变化**。 以前用 ComfyUI 搭 workflow；视频模型达到 SOTA 后被全部替换，工作从拼 workflow 变成写 prompt。
- **中美对比（个人观点）**。 视频生成领域中国明显领先，美国最接近的是 Gemini，但差距很大。

AI 生图里的汉字问题。 生成图片里的汉字经常出错。讨论中有人归因于 UTF-8 编解码，这个说法不太准确。更常见的解释是文本编码器缺少字符级的字形信息，以及训练数据里清晰的汉字渲染样本太少；汉字笔画复杂会放大这个问题。另外，专业画师目前仍然能识别出 AI 生成的图。

---

6. Data infra 与存储

6.1 多模态 AI data pipeline 的技术栈

搭一条完整的多模态 AI 数据管线，常被点名的组件有：

组件	角色
Ray / Spark	分布式计算与调度
Daft	Rust 实现的分布式查询引擎，原生支持多模态
LanceDB / Lance	多模态数据存储与检索，大厂内部有封装使用
Parquet	列式存储格式
Paimon / Iceberg	湖仓表格式
CDC（如 DTS 2.0）	数据同步
BaaS / Serverless 数据层	托管数据服务

相关的参考方向包括：多模态数据管道、OCR/ASR/VLM 处理、多模态 RAG、Lance + Ray、Iceberg + Ray 多模态湖仓、Hugging Face 多模态数据管道。有资料称 Flotilla 比 Spark / Ray Data 快 2 到 18 倍（未验证）。LanceDB 的一篇介绍：https://lqhl.me/blog/lancedb/ 。

一个判断是：这类资源里大部分是老一代数据湖迭代过来的，真正需要新学的不多。

6.2 CDC 和 BaaS 其实就是数据同步

CDC 和 BaaS 常被当作 AI 数据管线的关键组件，但实践者的看法是：它们本质上就是数据同步，换成别的云服务也一样。几种实际做法：

- MySQL binlog 走 Flink CDC，直接写入 HBase。这套方案技术含量不高，主要问题在稳定性。
- 数据直接落到对象存储（OSS），由 Kafka 消费，再进入离线处理。
- 也有团队用过 Flink CDC，后来切掉了。

6.3 Data infra 的前景

对 data infra 的前景有三种判断：

- 存储 infra 是永远需要的。
- infra 是周期性的。
- 数据本身肯定有前景，data infra 不确定。

6.4 数据库内核和 agent

一个观点是：数据库内核很难被 agent 自动化替代。它有状态，一出故障就直接不可用，而且是分布式系统。相比之下，算子可以在单机上反复 loop，失败了大不了重新训练。

这个观点还认为数据库内核的难度比训练高，甚至比 Linux 内核还难，因为涉及 GPU Direct 和存储层的寄存器开发；而 Linux 的文件系统本身就是数据库背景的人做的。这一说法存在争议。

国内数据库方向的公司包括 TiDB、OceanBase（架构不同于其他家）、AutoMQ、ScopeDB、BlueDB 等。

6.5 DB infra 和 SRE

关于 infra 的护城河在哪里，有两种看法：

- 在 SRE。DB infra 已经很完善，运维稳定性才是核心。
- 在 DB infra。没有 DB infra，数据都没地方存。

另外，日志分析、事故排查这类工作，除了数据安全方面的顾虑，技术上 AI 都已经能做。

6.6 存储实践

- **图片不要存进关系库。** 不要把图片转成 base64 存表。正确做法是文件存对象存储，库里只存链接；对外访问时再加 token 和过期时间。
- **业务大数据走数据湖逻辑。**

- **离线集群利用率**。 离线集群利用率在 60% 左右时，可以用无限量的数据处理任务把它打满。
- **业务表设计**。 数据量不大的 CRUD 表，直接加字段即可，不必过度设计。
- **测试数据**。 开发自测时造测试数据是一大痛点（这是业务测试数据，不是训练语料）。
- **分析工具的局限**。 通用 coding agent 目前还做不好复杂的数据分析。
- **第三方工具的数据协议**。 有的 AI 编程工具的数据协议会保存用户数据 30 天，因此企业管理员常常禁用这类工具。

---

7. 具身智能与机器人数据

7.1 数据是具身的第一瓶颈

国内具身数据普遍存在三个问题：

1. 大量采集数据是约五倍慢动作，不像真人的动作节奏。
2. 奖励信号是错的。
3. 用这样的轨迹训出来的模型一定有问题。

另一个方向是用人脑信号给机器人数据打标签，因为普通视频喂不出通用机器人。难度可以类比：只清洗 26 个字母的手写数据，清洗代码就要三四千行，视觉数据只会更复杂。

7.2 行业现状

具身方向还远未收敛，大约 3 个月出一个版本，经常推倒重来，几乎没有完全跑通的方案。一个较务实的迭代模式是：

```
收集现场数据 → 归因 → 修复 → 再部署
```

岗位上，具身相关招聘通常包括算法、运控、VLA、数采。中小具身公司的数据工程师往往一个人全包，从采集到管线都做。名为「平台」的岗位，实际做的常常是 data infra。

自动驾驶方向同样是数据驱动，拥有更多真实出行数据的一方有明显优势。

7.3 世界模型数据采集

世界模型数据的一种采集方式是：把手机挂在胸前，以慢速行走拍摄第一人称视频。

7.4 机器人数据上报的容量规划

一个典型场景：机器人每 500ms 上报一条数据，每天开机 8 小时，多台机器同时运行时，数据量会迅速膨胀。

协议。 有人会想到 MQTT，但 MQTT 是 IoT 协议，机器人场景未必适用。实际使用的协议讨论中没有给出。

量级估算。 两种估算差别很大：

- 按 1 秒 2 条、8 小时计算，单台只有万级。
- 实际生产环境中，多台合计可能一天几百万条；随着机器人数量增加，量级还会继续上涨。

处理策略。

- 先判断数据是否有长期价值，有用就存。
- 容量不够就扩容。
- 用 MySQL 分区表，并定时清理旧分区。
- 冷数据上传到对象存储（如 COS）。
- 数据主要用于查询时，可以定期清理。

机器人规模化之后存储成本会显著上升，所以也有观点认为具身场景需要专门的数据库。

---

8. 广告与业务数据

8.1 推荐训练数据 pipeline

推荐系统的训练数据管线通常包括：

- 特征生成
- 负采样
- 序列构建
- 多模态特征接入
- 线上线下一致性

面向大模型推荐的数据计算和存储，常用技术栈是 Flink、Spark、Ray，配合 Paimon 或 Iceberg。推理侧则需要建设多模态数据生产链路，用于广告素材理解、内容生成和投放优化。「基于模型的内容理解工程」实际对应多模态检索或图搜图。

8.2 推荐模型怎么选

传统机器学习在很多推荐场景下已经够用。模型选择主要按数据量来：从 FM、DeepFM 到 DNN、DCN，然后做特征工程。常见的迭代手段就是加长序列、加特征、做特征交叉组合。

8.3 AI 素材标签与投放归因

一个 AI 广告数据系统的典型设计如下。

素材标签体系：标签示例包括口播、剧情、混剪、前 3 秒结构等。

归因分析：用投放数据分析标签对 CTR、CVR、ROI 的影响，再反推素材该怎么做。

多维数据：账户、计划、素材、时段、人群。

配套能力：A/B 实验和数据看板。

技术栈：MySQL 为基础，Hive、Spark、ClickHouse 为加分项。

8.4 联盟广告的数据共享边界

多个媒体接入同一家广告联盟后，数据共享有明确边界：

- 曝光、点击这类投放数据会回传。
- 任何涉及用户的数据都不共享。

8.5 数据隔离与自建 infra

同一集团内不同广告业务之所以会各自自建 infra，一个重要原因是数据源不同、需要数据隔离。当预算、合同或业务主体不一致时，数据不能直接进入另一条业务链路，需要通过其他平台绕开主体不一致的问题。等主体和数据体系合并之后，这类隔离才会逐步消除。

8.6 广告引擎的链路追踪

广告引擎做全链路 trace 的难点在于数据量。请求量太大时，全量写入 ClickHouse 的成本太高，所以很多引擎并没有做 trace。

8.7 其他业务数据经验

- **搜索闭环**：在聚合页做「看后搜」「点后搜」，在合适时机给出搜索推荐词，把用户的划走或返回转化成继续搜索，从而提升消耗。
- **电商闭环广告**：按订单结算转化（conv）。不同平台的点击成本差异可能达到 20% 到 30%，有的高出一半。
- **广告分布**：用户看到某一家的广告特别多，主要原因不是标签，而是预算。预算高的广告主会铺量。
- **分机型出价**：高端安卓机型的投放价格可能高于 iOS。
- **数据驱动架构**：缺乏数据敏感性会错过收益，归因和推荐链路上可以建模拿收益。
- **出价与计价**：RPM 引擎优化、博弈出价、CTR 模型、计价转换是广告引擎侧的核心技术点。
- **数据孤岛**：打通营销、数据分析、广告投流、回收数据和 agent 操作，是一类数据产品的思路。

8.8 金融数据

金融类 agent 的关键是看数据来源。金融数据库多为闭源，Wind 等是主要数据源。

9. 岗位与行业观察

这一节是从业者的个人观点，分歧较大，仅供参考。

9.1 加班和薪资

- **一种主观排序。** 加班强度从高到低是数据管线、数据算法、训练 infra、大模型算法。持这个看法的人认为工资大致也是这个顺序。
- **数据算法的薪资。** 一种看法认为数据算法的包和业务算法差不多；另一种说法是，同一类「基模算法数据方向」岗位，在有的公司和训练岗差不多，在有的公司能差两倍。业务算法的毕业包也有百万量级。
- **数据管线最辛苦。** 普遍认为数据管线被催得最狠。

9.2 算法和 infra

- 一种看法认为算法强于 infra：包更大；infra 最多是工程师，算法可以成为科学家。
- 另一种表述是：基模的数据算法强于 infra，而且这里的数据算法其实就是负责数据方向的基模算法。
- 业务 vs 支撑：算法是业务，infra 是支撑部门，离业务越近越好，这和 Java 时代中台与业务的关系是一个道理。
- 由此延伸的类比：AI Data 就是大模型领域的「业务开发」，相当于互联网体系里中台后端和广告业务的关系。

9.3 竞争力从哪来

- **一个人接一条线。** 想有竞争力，最好独立负责一整条线。目前比较容易接手的是从数据工程到数据算法的完整链路。
- **反例。** 不敢单独接活、只做高性能计算优化的人，会担心自己缺乏竞争力。
- **生态位比工作内容重要。** 现在能不能留下来，不取决于你在做什么，而取决于这件事处在什么生态位、是不是创新位。工作内容本身已经不能作为是否会被淘汰的依据。
- **一个经历。** 有人入职一个月就负责 omni 模型方向并参与视频模型的 RL，在技术报告里的排位超过了干了半年的 infra 负责人。原因是这个方向本来没人，被分给了他。

9.4 行业动态（未经核实）

据群内转述，DeepSeek 开始招 skills 工程师：招编译器等底层系统方向的 infra 专家，只负责写 skills，理由是「专家才写得好 skills」。转述者推测，按 DeepSeek 的开源文化，这些 skills 写完后大概率会开源。

10. 待解的技术问题

讨论中提出但没有答案的问题：

- 数据工程的 infra 是否应算作 AI infra；数据侧对上层应该提供什么样的封装。
- 预训练现在具体包含哪些阶段。
- trick data 具体是哪些数据，为什么放在 mid-train。
- 评测集混入数据的比例、标注规范、rubric 的细则。
- code data 怎么清洗。
- 视频打标具体用什么模型。
- 机器人数据上报用什么协议。
- 世界模型数据的格式，具身数采的具体方法。
- 多模态 OCR 场景下「识别」与「理解」的边界。
- agentic data 怎么做；用自己清洗的数据做一次预训练是否可行。
- 「数据算法」和「基模算法数据方向」是否是同一岗位，各团队划分不一。
- 视频生成模型之间的优劣（minimax-h3、wan2.2、sd2.5 等），各方评测结论不一致。
- 数据领域是否已经收敛。一种判断方法是看有没有成熟的「高端词汇」：词汇成熟说明已被研究透，超额收益在尚未收敛的问题上。对数据领域是否已有这类词汇，看法完全相反。

12. 参考资料

12.1 学习资料

一种学习建议是：先拿数据工程的书当「导览图」，再按自己所在的垂域和子分支深挖；实际工作用到的东西可能和通用资料完全不搭，最好去实习看看业界怎么做。

- 大模型数据工程（适合当导览图）：https://datascale-ai.github.io/data_engineering_book/
- CS336 从零构建语言模型（中文导读）：https://feigaobox10.github.io/llm-from-scratch-reader/%E4%BB%8E%E9%9B%B6%E6%9E%84%E5%BB%BA%E8%AF%AD%E8%A8%80%E6%A8%A1%E5%9E%8B%C2%B7CS336%E9%80%9A%E8%AF%86%E8%AF%BE.html#sec1
- 长程 Agent 的 GRPO 训练：https://github.com/qiqihezh/agentic-grpo-longhorizon
- https://yyhdbl.github.io/
- AI Agent 书：https://github.com/bojieli/ai-agent-book
- 以数据为中心的 AI 入门：https://github.com/OpenDCAI/DCAI-Starter-Guide
- LLM Training Lab 21：训练中途 loss 突然飙升的处理（混合精度、梯度监控与恢复策略）

12.2 论文与工具

- Chinchilla：https://arxiv.org/abs/2203.15556
- Aspire：arXiv 2608.31111
- S^3Gym：arXiv 2608.31100

- HarnessDev：arXiv 2609.01437
- Awesome-Data-Centric-Recursive-Improvement：https://github.com/zoezou2015/Awesome-Data-Centric-Recursive-Improvement
- LanceDB：https://lqhl.me/blog/lancedb/
- 图片去重工具：Meta PDQ、SSCD、CLIP、Qwen reranker
- Agent 评测基准：SWE-bench、Terminal-Bench、GAIA、τ-bench
- OCR：PaddleOCR
- 视频生成模型：minimax-h3、wan2.1 / wan2.2、Seedance 2.0 / 2.5、ComfyUI